

Angular 21 Schulungs-Glossar (Tag 1 – Tag 3)

Die 5 zentralen Kern-Metaphern des Kurses

Konzept	Bildhafte Metapher für DGS	Technische Bedeutung
Component (Komponente)	LEGO-Baustein / Box	Ein sichtbarer, in sich geschlossener Teil der Webseite (z. B. Header, Kundenliste, Kundenkarte). Besteht aus Aussehen (HTML), Verhalten (TypeScript) und Design (CSS).
Signal (Reaktiver Zustand)	Lämpchen / Sensor / Automatische Glocke	Ein Datenbehälter. Sobald sich der Inhalt ändert, „leuchtet“ er auf und die HTML-Seite aktualisiert den Text an dieser Stelle vollautomatisch, ohne die ganze Seite neu zu laden.
Service & Dependency Injection	Spezialwerkzeug / Fachabteilung auf Abruf	Eine Hilfsklasse für Geschäftslogik (z. B. Daten vom Server holen). Komponenten müssen das Rad nicht neu erfinden, sondern „bestellen“ den Service per <code>inject()</code> .
Routing	Wegweiser / Umschalten der Bühne	Die Webseite bleibt immer dieselbe (kein Browser-Reload), aber je nach URL (z. B. <code>/customer/5</code>) wird ein anderer Baustein auf die Bühne geholt.
Observable / Stream	Fließband / Daten-Strom	Daten, die mit zeitlicher Verzögerung nacheinander eintreffen (z. B. Antworten vom Webserver oder Klicks des Nutzers).



17 TAG 1: Grundlagen, Standalone-Architektur, Templates & Signals

Themenschwerpunkt: *TypeScript, Projektstruktur, Komponenten erstellen, Daten im Template anzeigen, Bedingungen/Schleifen und lokaler Zustand mit Signals.*

1.1 TypeScript & Tooling

- **TypeScript (TS):** Eine Erweiterung von JavaScript. Der Code wird streng typisiert (jede Variable hat einen festen Typ wie Zahl, Text oder Wahrheitswert), um Tippfehler vor dem Ausführen zu verhindern.
- **Typisierung (`string`, `number`, `boolean`, `Date`):**
 - `firstname: string` = Nur Text erlaubt.
 - `age: number` = Nur Zahlen erlaubt.
 - `isActive: boolean` = Nur `true` (wahr) oder `false` (falsch).
- **Interface / Modell (`interface Customer { ... }`):** Ein Bauplan für Datenobjekte. Definiert, welche Felder ein Kunde haben muss (z. B. Vorname, Nachname, ID).
- **CLI (Command Line Interface / Terminal):** Das Text-Befehlswerkzeug. Mit Befehlen wie `ng serve` (Entwicklungsserver starten) oder `ng build` (fertige App bauen) wird Angular gesteuert.
- **Node.js & npm:** Die Ausführungsumgebung auf dem Entwicklerrechner und der Paketmanager, der benötigte Bibliotheken aus dem Internet lädt.

1.2 Standalone Components & Bootstrapping

- **Standalone Component (`standalone: true`):** Moderne Angular-Komponenten sind selbstständig. Sie benötigen keine übergeordneten Module (`NgModule`) mehr und deklarieren ihre benötigten Abhängigkeiten

direkt im eigenen `imports: []`-Array.

- **Bootstrapping** (`bootstrapApplication(AppComponent)`): Der Zündschlüssel beim Starten der App in `main.ts`. Lädt die Wurzelkomponente (`AppComponent`) in den Browser.
- **Decorator** (`@Component({ ... })`): Ein Hinweisschild vor einer Klasse, das Angular sagt: „Diese Klasse ist eine UI-Komponente mit HTML-Vorlage und Stylesheet“.
- **Selector** (`selector: 'app-root'`): Der eigene HTML-Tag-Name, unter dem die Komponente in anderen Vorlagen verwendet werden kann (z. B. `<app-root></app-root>`).
- **Zoneless** (`provideZonelessChangeDetection()`): Das moderne Angular-System zur Aktualisierung der Bildschirmanzeige rein über Signals, ohne alte Hintergrund-Überwachung (`zone.js`).

1.3 Templates & Datenbindung (Data Binding)

- **Template (HTML-Vorlage)**: Die sichtbare Struktur der Komponente.
- **Interpolation** (`{{ customer.firstname }}`): Text-Ausgabe von Variablen im HTML. Bild: *Einsetzen eines Platzhalters in einen Formularbrief.*
- **Property Binding** (`[src]="customer.imageUrl"`): Übergabe von dynamischen Werten aus dem TypeScript-Code an HTML-Attribute (gekennzeichnet durch eckige Klammern `[...]`).
- **Event Binding** (`(click)="onSelect()"`): Reaktion auf Nutzeraktionen wie Mausklicks oder Tastatureingaben (gekennzeichnet durch runde Klammern `(...)`).
- **Two-Way Binding** (`[(ngModel)]="customer.firstname"`): Beidseitige Datenverbindung. Ändert der Nutzer den Text im Eingabefeld, ändert sich die Variable im Code – und umgekehrt. (Merkwort: *Banana in a Box* `[]()`).

1.4 Moderner Built-in Control Flow

- `@if (condition) { ... } @else { ... }`: Bedingte Anzeige. Ein HTML-Block wird nur gerendert, wenn die Bedingung wahr ist (z. B. Daten vorhanden).
- `@for (item of items; track item.id) { ... }`: Schleife/Wiederholung. Wiederholt einen HTML-Abschnitt für jedes Element einer Liste.
- `track`: Ein zwingend erforderlicher Schlüssel (z. B. Kunden-ID), damit Angular bei Änderungen nur genau das veränderte Listenelement neu zeichnet und nicht die ganze Liste (hohe Performance).
- `@empty`: Wird innerhalb von `@for` angezeigt, wenn die Liste leer ist (z. B. „Keine Kunden gefunden“).
- `@switch (status) { @case ('active') { ... } }`: Fallunterscheidung für mehrere feste Zustände.

1.5 Signals (Das reaktive Herzstück)

- `signal(initialValue)`: Erzeugt einen reaktiven Datenwert. Auslesen erfolgt mit Funktionsaufruf: `mySignal()`.
- `.set(newValue)`: Überschreibt den Wert eines Signals komplett.
- `.update(fn)`: Berechnet einen neuen Wert basierend auf dem alten (z. B. `count.update(c => c + 1)`).
- `computed(() => ...)`: Ein abgeleiteter Wert (Berechnung), der sich automatisch neu berechnet, sobald sich ein darin verwendetes Signal ändert (z. B. `fullName = computed(() => firstname() + ' ' + lastname())`).
- `effect(() => ...)`: Führt automatisch eine Nebenwirkung aus (z. B. Logging oder Speichern im LocalStorage), wenn sich beobachtete Signals ändern.

1.6 Komponenten-Kommunikation

- **Signal Input** (`input<T>()` / `input.required<T>()`): Datenempfang von der Elternkomponente. Bild: *Der Briefkasten der Kindkomponente.*

- **Output API** (`output<T>()`): Sender für Ereignisse an die Elternkomponente. Ersetzt das frühere `EventEmitter`. Aufruf mit `.emit(daten)`.
- **Deferrable Views** (`@defer (on viewport | on idle)`): Verzögertes Laden von schweren Komponenten (z. B. große Bilder oder Diagramme), erst wenn der Nutzer an die entsprechende Stelle der Seite scrollt.

17 TAG 2: Formulare, Services, DI, Lifecycle, Pipes & RxJS

Themenschwerpunkt: *Datenbearbeitung, Geschäftslogik in Services auslagern, Datenformatierung mit Pipes und asynchrone Datenströme.*

2.1 Template-Driven Forms (Einfache Formulare)

- **FormsModule**: Das Angular-Modul für einfache Formulare.
- **[(ngModel)]**: Bindet Eingabefelder direkt an Eigenschaften eines Objekts.
- **Form Submission** (`(ngSubmit)="onSave()"`): Das Absenden eines Formulars durch Klick auf einen Speichern-Button.

2.2 Services & Dependency Injection (DI)

- **Service**: Eine TypeScript-Klasse, die reine Logik und Datenverwaltung enthält (z. B. `CustomerService`). Komponenten sollen schlank bleiben und delegieren Aufgaben an Services.
- **@Injectable({ providedIn: 'root' })**: Kennzeichnet eine Klasse als injizierbaren Dienst, der app-weit als Singleton (nur eine einzige Instanz existiert) verfügbar ist.
- **Dependency Injection (DI)**: Entwurfsmuster, bei dem eine Komponente benötigte Dienste nicht selbst mit `new Service()` erzeugt, sondern vom Angular-System automatisch „überreicht“ bekommt.
- **inject(CustomerService)**: Die moderne Angular-Funktion, um Dienste direkt bei der Variablen-Deklaration anzufordern (Alternative zum alten Konstruktor-Parameter).

2.3 Component Lifecycle (Lebenszyklus)

- **Lifecycle Hooks**: Spezielle Methoden, die Angular zu bestimmten Zeitpunkten im Leben einer Komponente automatisch aufruft.
- **OnInit / ngOnInit()**: Wird genau einmal aufgerufen, nachdem die Komponente initialisiert und alle Inputs bereitgestellt wurden. Typischer Ort zum ersten Laden von Daten.
- **OnDestroy / ngOnDestroy()**: Aufräumarbeiten kurz bevor die Komponente vom Bildschirm entfernt wird (z. B. Timer stoppen oder Streams beenden).

2.4 Pipes (Datenformatierung im Template)

- **Pipe (Symbol |)**: Ein Filter/Transformator im HTML-Template, der Rohdaten für den Benutzer formatiert darstellt, ohne die Originaldaten im Speicher zu verändern.
- **DatePipe** (`customer.lastOrderDate | date:'mediumDate'`): Formatiert ein technisches Datum in ein lesbares Format (z. B. `10.10.2026`).
- **CurrencyPipe** (`price | currency:'EUR'`): Formatiert Zahlen als Geldbetrag mit Währungssymbol.
- **Custom Pipe** (`@Pipe({ name: 'hasLikes', standalone: true })`): Eine selbst geschriebene Pipe mit einer `transform(value)`-Methode für eigene Formatierungs- oder Filterlogik.

2.5 Asynchronität, RxJS & Signals-Brücke

- **Asynchron**: Vorgänge, die Zeit benötigen und im Hintergrund laufen (z. B. Serveranfragen). Der Browser blockiert dabei nicht.
- **Observable**: Ein Datenstrom aus der Bibliothek **RxJS**. Liefert 0, 1 oder viele Werte über die Zeit.

- `subscribe()` & `unsubscribe()` : Anmelden am Datenstrom (Zuhören) und Abmelden (um Speicherlecks zu verhindern).
- `toSignal(observable$)` : Die moderne Brückenfunktion in Angular. Wandelt ein RxJS-Observable direkt in ein Angular-Signal um, sodass im Template keine manuelle `async` -Pipe mehr nötig ist.

17 TAG 3: Routing, Reaktive Formulare & HTTP-Backend-Anbindung

Themenschwerpunkt: *Mehrseitige Anwendungen (SPA-Routing), typsichere Validierungsformulare und echte REST-API-Kommunikation.*

3.1 Routing & Single Page Application (SPA)

- **Single Page Application (SPA)**: Eine Webanwendung, die nur aus einer einzigen HTML-Seite besteht. Beim Wechsel von Seiten/Ansichten wird die Webseite nicht neu geladen, sondern Angular tauscht nur Komponenten blitzschnell im Browser aus.
- `provideRouter(routes, withComponentInputBinding())` : Aktiviert das Routing-System in `main.ts`.
- **Routes (`app.routes.ts`)**: Eine Konfigurationsliste, die URL-Pfade mit Komponenten verknüpft (z. B. `{ path: 'customer', component: CustomerListComponent }`).
- `<router-outlet></router-outlet>` : Der dynamische Platzhalter/Rahmen im HTML, an dessen Stelle die zur aktuellen URL passende Komponente gerendert wird.
- `routerLink="/customer"` : Das Angular-Äquivalent zum normalen HTML-Link (`<a href>`), verhindert das Neuladen der Seite.
- **Dynamic Routes (`path: 'customer/:id'`)**: Routen mit variablem Parameter (z. B. ID des Kunden).
- **Component Input Binding**: Durch `withComponentInputBinding()` wird der URL-Parameter `:id` direkt als Signal-Input `id = input<string>()` in die Zielkomponente injiziert.

3.2 Strictly Typed Reactive Forms (Reaktive Formulare)

- **Reactive Forms (`ReactiveFormsModule`)**: Ein robuster, code-getriebener Ansatz für komplexe Formulare mit voller TypeScript-Typsicherheit.
- `FormControl<T>` : Verwaltet den Wert, den Gültigkeitsstatus (Valid/Invalid) und Benutzerinteraktionen (Touched/Dirty) eines einzelnen Eingabefeldes.
- `FormGroup` : Bündelt mehrere `FormControl` -Felder zu einem gemeinsamen Formularobjekt.
- `FormBuilder` / `NonNullableFormBuilder` : Ein Hilfsdienst (`fb = inject(FormBuilder)`), um Formulare mit weniger Schreibaufwand zu erstellen.
- **Validierung (`Validators.required` , `Validators.minLength`)**: Prüfregeln für Eingabefelder (z. B. Pflichtfeld oder Mindestlänge).

3.3 HTTP & REST-Backend-Kommunikation

- **REST API**: Eine standardisierte HTTP-Schnittstelle auf dem Webserver zum Austausch von JSON-Daten.
- **HTTP-Methoden**:
 - `GET` : Daten vom Server abrufen (z. B. `GET /api/customer`).
 - `POST` : Neue Daten auf dem Server erstellen (z. B. `POST /api/customer`).
 - `PUT` / `PATCH` : Bestehende Daten aktualisieren.
 - `DELETE` : Daten auf dem Server löschen.
- `provideHttpClient()` : Stellt den `HttpClient` -Dienst in der gesamten App bereit.
- `HttpClient` : Der Angular-Dienst zum Senden von Netzwerkanfragen. Methoden wie `.get<Customer[]>()` geben ein RxJS-Observable zurück.

- **JSON (JavaScript Object Notation):** Das Textformat, in dem Daten zwischen Server und Browser übertragen werden.
- **Dev-Proxy (`dev.proxy.json`):** Ein Entwicklungs-Werkzeug, das Anfragen von `localhost:4200` (Angular) an `localhost:3000` (Backend-Server) weiterleitet, um Browser-Sicherheitsblockaden (CORS) zu vermeiden.



Alphabetisches Schnell-Nachschlagewerk (Quick Lookup)

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
AppComponent	<i>Äpp-Komponent</i>	Die Hauptkomponente (Wurzel) der gesamten Angular-Anwendung.
Banana in a Box	<code>[()]</code>	Schreibweise für zweiseitige Datenbindung bei <code>[(ngModel)]</code> .
Binding	<i>Bain-ding</i>	Die Verknüpfung zwischen TypeScript-Variable und HTML-Anzeige.
bootstrapApplication	<code>bootstrapApplication(...)</code>	Startfunktion von Angular beim Laden der Webseite.
computed	<code>computed(() => ...)</code>	Berechnetes Signal, das automatisch aktualisiert wird.
Customer	<i>Kastemer</i>	Kunde / Datenmodell für Kunden in den Schulungsübungen.
DatePipe	<i>Deit-Paip</i>	Formatiert Datumsangaben im HTML lesbar.
Dependency Injection	<i>DI</i>	Automatisches Bereitstellen von Diensten und Klassen.
Event	<i>I-went</i>	Ereignis im Browser, z. B. Mausklick oder Tastendruck.
FormGroup	<code>FormGroup</code>	Gruppe von Formularfeldern in reaktiven Formularen.
HttpClient	<code>HttpClient</code>	Angular-Dienst für Kommunikation mit Web-Servern.
inject	<code>inject(...)</code>	Funktion zum Anfordern eines Dienstes per Dependency Injection.
input	<code>input()</code>	Empfängt Daten von der übergeordneten Elternkomponente.
Interpolation	<code>{{ ... }}</code>	Ausgabe von Variablen direkt im HTML-Text.
ng serve	<i>en-dschi sörw</i>	Terminal-Befehl zum Starten des lokalen Entwicklungsservers.
ngOnInit	<code>ngOnInit()</code>	Lebenszyklus-Methode beim Initialisieren einer Komponente.
Observable	<i>Ob-sör-wa-bl</i>	Datenstrom, der über die Zeit Werte liefert (RxJS).
output	<code>output()</code>	Sendet Ereignisse von der Kind- an die Elternkomponente.
Pipe	<code> </code>	Formatiert Daten direkt im HTML-Template.
Reactive Forms	<i>Ri-aktiv Forms</i>	Typsichere, im TypeScript-Code definierte Formulare.
RouterOutlet	<code><router-outlet></code>	Platzhalter im HTML, wo die geroutete Seite erscheint.

Fachbegriff	Aussprache / Code	Kurzerklärung (1 Satz)
Routes	<i>Rauts</i>	Liste der URL-Pfad-Zuordnungen der Anwendung.
Service	<i>Sör-wis</i>	Klasse für Geschäftslogik und Datenabruf.
Signal	<code>signal()</code>	Reaktiver Datenspeicher mit automatischer Benutzeroberflächen-Aktualisierung.
Single Page App	<i>SPA</i>	Webseite, die ohne Neuladen blitzschnell Seiten austauscht.
Standalone	<i>Ständ-ä-lon</i>	Selbstständige Komponenten ohne <code>@NgModule</code> .
Template	<i>Täm-pleit</i>	Die HTML-Vorlage einer Komponente.
toSignal	<code>toSignal(...)</code>	Wandelt ein RxJS-Observable in ein Angular-Signal um.
track	<code>track item.id</code>	Pflicht-Identifikator in <code>@for</code> -Schleifen für Performance.
TypeScript	<i>Teip-Skript</i>	Programmiersprache von Angular mit strenger Typisierung.
Zoneless	<i>Sohn-les</i>	Moderner Signal-Modus ohne veraltete <code>zone.js</code> -Überwachung.